ASSIGNMENT 4 – TASK 2

Spoken Digit Classification Using Recurrent Neural Networks, LSTM and GRU

Student Name: Sushmita Halder, Jyoti, Rajeev Kumar
Roll Number: 2621MA08, 2621MA12, 2621MA01
Group: 08 **Course:** Large Language Models
Submission Date: 13/10/2026

1. Introduction

This task classifies spoken Arabic digits (0–9) from the UCI Spoken Arabic Digit dataset using recurrent neural networks. Each utterance is a variable-length sequence of frames, and each frame is a vector of 13 MFCC (mel-frequency cepstral coefficient) features. Vanilla RNN, LSTM, and GRU classifiers are compared over five hyperparameter configurations using training-loss curves, validation performance, test results, a classification report, confusion matrices, and MFCC heatmaps.

2. Objectives

- Load the variable-length MFCC utterances, split them into training, validation, and test sets, and standardize the features.
- Handle variable sequence lengths with padding and packed sequences.
- Train and compare Vanilla RNN, LSTM, and GRU models under several hyperparameter configurations, using the training-loss convergence rule from the assignment.
- Report average training-error curves and a hyperparameter accuracy summary.
- Evaluate the validation-selected best model with a classification report and confusion matrix, and compare the confusion matrices of the best RNN and best LSTM.
- Visualize MFCC heatmaps of a representative utterance for every digit.
- Discuss performance and convergence of the three recurrent architectures.

3. Dataset and Preprocessing

The data are the UCI Spoken Arabic Digit text files (`Train_Arabic_Digit.txt` and `Test_Arabic_Digit.txt`), in which utterances are separated by blank lines and each line is one frame of 13 MFCC values. Labels are reconstructed from the documented block order (660 training and 220 test utterances per digit), and the loader checks the expected counts. According to the UCI documentation quoted in the notebook, the test speakers differ from the training speakers, so the supplied test file is kept separate from training and validation. A stratified 15% of the training file forms the validation set (`random_state = 42`). MFCCs are standardized per coefficient using the mean and standard deviation of the frames in the training split only.

Property	Value
Dataset	UCI Spoken Arabic Digit
Classes	10 (digits 0–9)
Frame features	13 MFCC coefficients per frame
Sequence length	Variable: 4 to 93 frames (minimum and maximum over train + test files)
Training file / test file	6,600 utterances (660 per digit) / 2,200 utterances (220 per digit)
Train / validation / test	5,610 / 990 / 2,200 utterances
Validation split	15% of the training file, stratified, random_state = 42
Standardization	Per-coefficient mean/std from training-split frames only
Random seed / device	42 / CPU

4. Sequential Representation

Each utterance is a sequence $X = [x_1, x_2, ..., x_T]$ with x_t in R^{13}, where the number of frames T varies from utterance to utterance. The recurrent layer reads the frames in time order and the final hidden state is used for classification.

4.1 Data loading and splitting output

Notebook output of the loading step (Section 2) and the split step (Section 3):

```
Using device: cpu
Train file exists: True
Test file exists: True
Training utterances: 6600
Test utterances: 2200
First training sequence shape: (38, 13)
Number of MFCC features: 13
Minimum sequence length: 4
Maximum sequence length: 93
Training class counts: [660 660 660 660 660 660 660 660 660 660]
Test class counts: [220 220 220 220 220 220 220 220 220 220]

Train split: 5610
Validation split: 990
Test split: 2200
Feature count: 13
```

4.2 Padding and packing

Because utterances have different lengths, each mini-batch is padded with zeros only up to the longest utterance in that batch (`pad_sequence`). The true lengths are passed to `pack_padded_sequence`, so the recurrent layer does not treat padded frames as speech. Notebook output for one training batch:

```
Batch input shape (batch, time, MFCC): torch.Size([64, 57, 13])
Sequence lengths in this batch: [30, 38, 39, 38, 37, 38, 33, 39, 40, 37]
Labels in this batch: [0, 5, 0, 5, 5, 7, 9, 5, 3, 0]
```

5. Qualitative Feature Visualizations

Figure 1 shows the standardized MFCC heatmap (time frame on the horizontal axis, MFCC coefficient on the vertical axis) of the first test utterance of every digit. The notebook produced these ten panels as one figure; they are shown here in four parts for legibility. The panel titles give the number of frames T of each utterance.

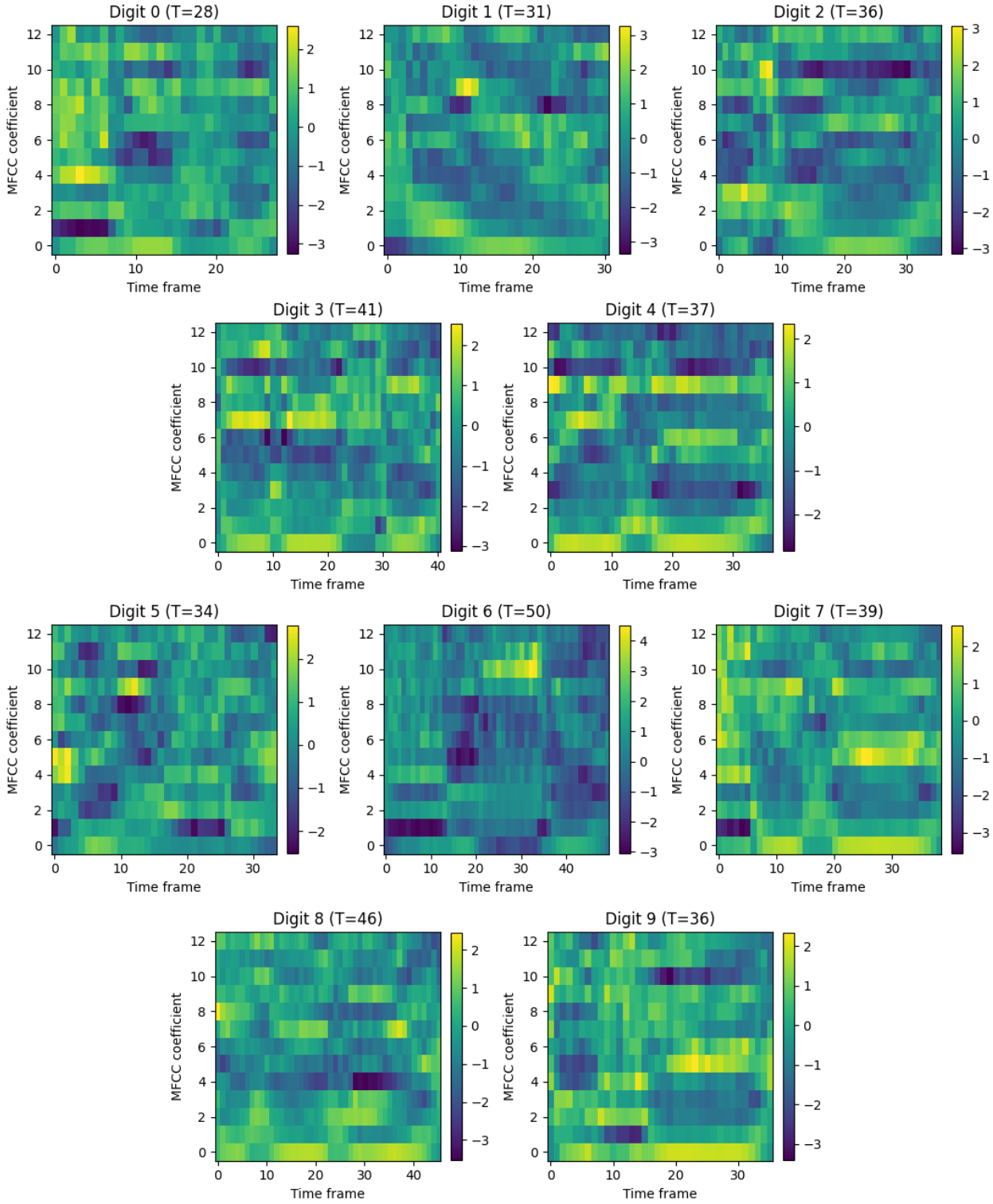


Figure 1. MFCC coefficient heatmaps for example test utterances: digits 0–2 and 3–4 (top two rows), digits 5–7 and 8–9 (bottom two rows).

6. Model Setup

Component	Configuration
Recurrent layer	nn.RNN (tanh), nn.LSTM, or nn.GRU; unidirectional, batch-first, packed input
Hidden size / layers	64 units × 1 layer, or 128 units × 2 layers (Table 1)
Dropout	0.0 for 1-layer models; 0.2 between the recurrent layers of 2-layer models
Output	Linear(hidden size, 10) on the final hidden state of the last layer
Optimizer / loss	Adam (0.001) / cross-entropy loss; gradient-norm clipping at 5.0
Batch size	64 utterances
Stopping rule	Primary: change in average training loss between consecutive epochs below 1e-4. Safety stops: validation loss not improved for 12 epochs; maximum 200 epochs
Weights kept	Weights of the epoch with the lowest validation loss are restored after training
Model selection	Best configuration chosen by validation accuracy (test set not used for selection)

Table 1. Explored configurations (all with learning rate 0.001, batch size 64). The parameter count is computed from the architecture (recurrent weights and biases plus the output layer).

Model	Cell	Hidden units	Layers	Dropout	Learning rate	Parameters
RNN_h64_layers1	RNN	64	1	0.0	0.001	5,706
RNN_h128_layers2	RNN	128	2	0.2	0.001	52,618
LSTM_h64_layers1	LSTM	64	1	0.0	0.001	20,874
LSTM_h128_layers2	LSTM	128	2	0.2	0.001	206,602
GRU_h64_layers1	GRU	64	1	0.0	0.001	15,818

7. Training Error and Convergence (a)

7.1 Average training error versus epochs

Figure 2 plots the average training cross-entropy loss against the epoch for every explored configuration. Each run stops at its own epoch, according to the stopping rules in Section 6.

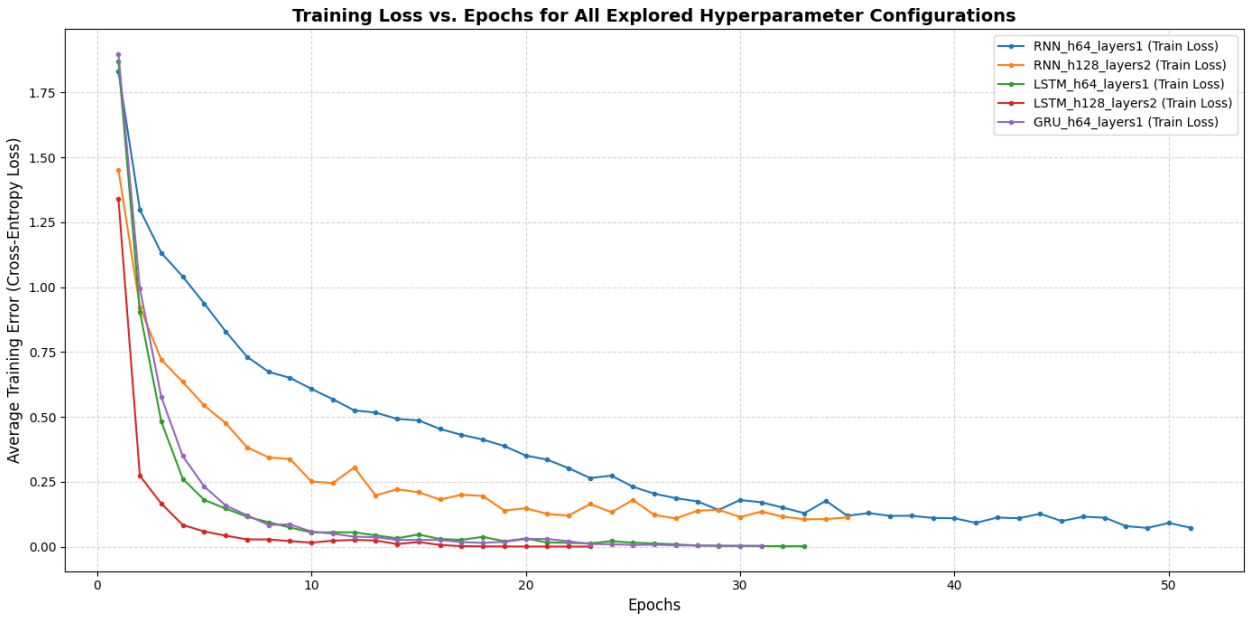


Figure 2. Average training error (cross-entropy loss) versus epochs for all five configurations.

Figure 3 shows the corresponding average validation loss.

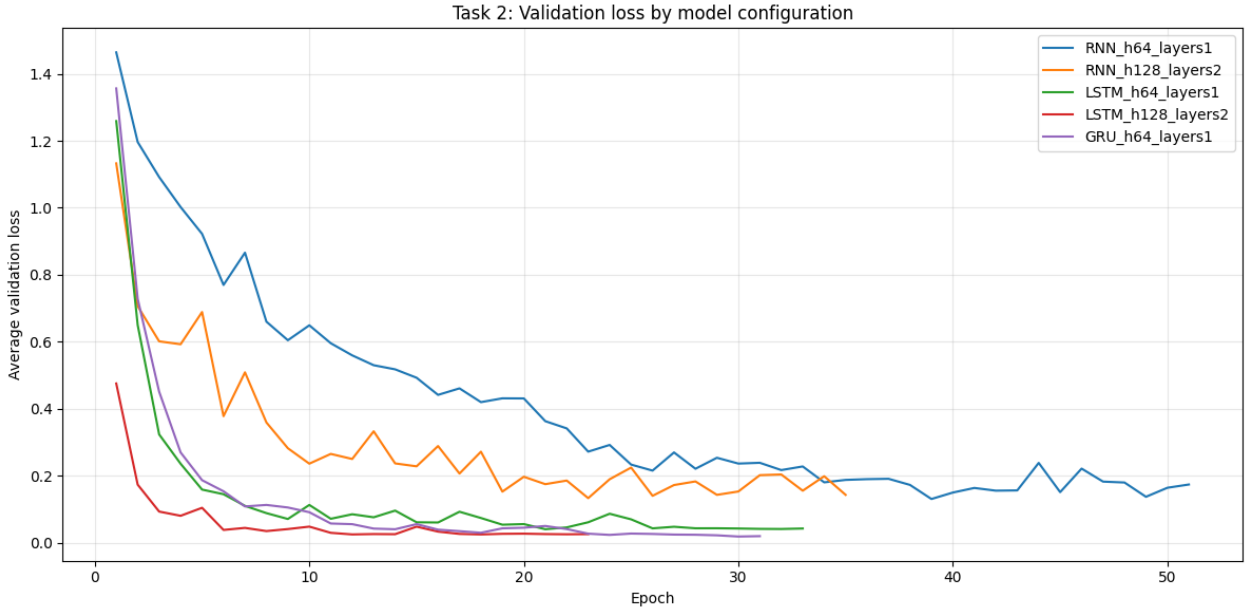


Figure 3. Average validation loss versus epochs for all five configurations.

Figure 4 re-plots the per-epoch training and validation accuracy from the training log (Appendix A).

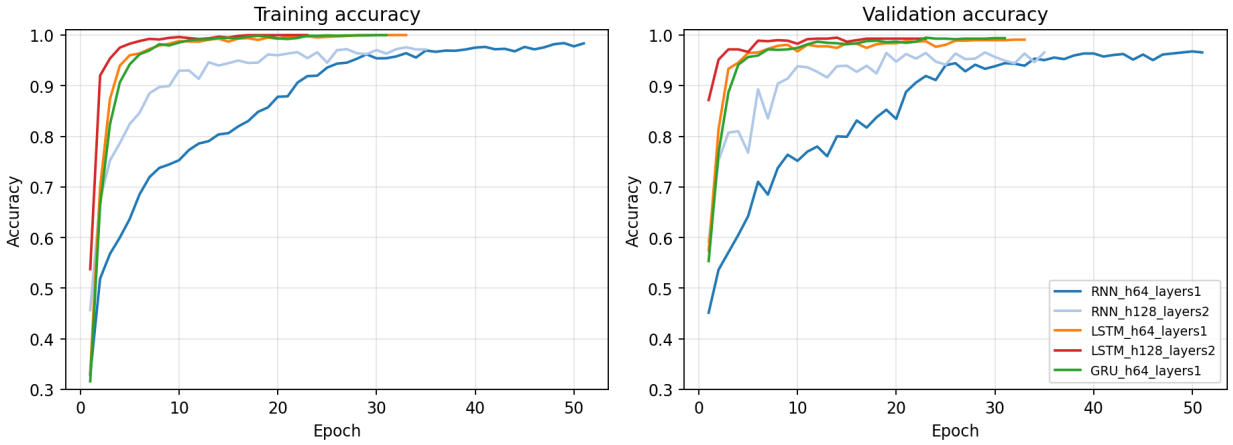


Figure 4. Training and validation accuracy per epoch (re-plotted from the notebook's training log).

7.2 Convergence summary

The table below is read from the per-epoch training log. The final-epoch values are those logged during training, while the “best epoch” is the epoch with the lowest validation loss, whose weights were restored. The last column gives the first epoch at which validation accuracy reached 95%.

Model	Epochs run	Best epoch	Train loss, epoch 1	Train loss, final	Val. loss, final	First epoch val. acc. at least 95%
RNN_h64_layers1	51	39	1.8326	0.0733	0.1742	34
RNN_h128_layers2	35	23	1.4534	0.1130	0.1433	19
LSTM_h64_layers1	33	21	1.8711	0.0018	0.0429	5
LSTM_h128_layers2	23	18	1.3410	0.0005	0.0261	2
GRU_h64_layers1	31	30	1.8996	0.0036	0.0201	5

Model	Reason the run stopped
RNN_h64_layers1	Early stop: validation loss did not improve for 12 epochs
RNN_h128_layers2	Early stop: validation loss did not improve for 12 epochs
LSTM_h64_layers1	Early stop: validation loss did not improve for 12 epochs
LSTM_h128_layers2	Convergence rule met: change in training loss = 8.49001e-05 (< 1e-4)
GRU_h64_layers1	Convergence rule met: change in training loss = 3.88723e-05 (< 1e-4)

8. Hyperparameter Accuracy Summary (b)

All five configurations were evaluated on the training, validation, and test sets after restoring the best-validation-loss weights. The table is sorted by validation accuracy, as in the notebook.

Model	Units	Layers	Epochs	Train acc. %	Val. acc. %	Test acc. %	Train loss	Test loss
GRU_h64_layers1	64	1	31	99.96	99.39	96.05	0.0037	0.1785
LSTM_h128_layers2	128	2	23	99.98	99.29	97.27	0.0010	0.1197
LSTM_h64_layers1	64	1	33	99.80	98.79	94.82	0.0114	0.2320
RNN_h128_layers2	128	2	35	97.38	96.46	91.91	0.0933	0.3445
RNN_h64_layers1	64	1	51	98.13	96.36	88.86	0.0768	0.5088

The notebook also saved this table as `task2_results.csv`. The best configuration by validation accuracy is GRU_h64_layers1 (99.39%). The highest test accuracy among the explored configurations is LSTM_h128_layers2 (97.27%), but the test set was not used to select the model.

9. Main Model Comparison

The best configuration of each cell type, chosen by validation accuracy, is compared below. The best RNN is RNN_h128_layers2, the best LSTM is LSTM_h128_layers2, and the best GRU is GRU_h64_layers1 (the only GRU configuration run). Validation loss is the lowest validation loss reached during training.

Model	Train acc. %	Val. acc. %	Train loss	Val. loss	Test acc. %	Test loss
Vanilla RNN (h128, 2 layers)	97.38	96.46	0.0933	0.1335	91.91	0.3445
LSTM (h128, 2 layers)	99.98	99.29	0.0010	0.0252	97.27	0.1197
GRU (h64, 1 layer)	99.96	99.39	0.0037	0.0190	96.05	0.1785

The validation-selected model is GRU_h64_layers1 with 99.39% validation accuracy and 96.05% test accuracy (2,113 of 2,200 test utterances correct).

10. Classification Reports and Confusion Matrices (c)

Each model below was evaluated on the same held-out test set of 2,200 utterances (220 per digit). Rows of each confusion matrix are actual digits and columns are predicted digits.

10.1 Validation-selected model: GRU_h64_layers1 – test accuracy 96.05%

Classification report as printed by the notebook (Section 9):

Class	Precision	Recall	F1-score	Support
0	0.90	0.97	0.93	220
1	0.95	1.00	0.97	220
2	0.94	0.99	0.96	220
3	0.97	0.92	0.94	220
4	0.99	0.96	0.97	220
5	0.98	0.98	0.98	220
6	0.98	0.85	0.91	220
7	0.95	0.95	0.95	220
8	0.97	0.99	0.98	220
9	0.99	1.00	0.99	220
accuracy			**0.96**	**2200**
macro avg	**0.96**	**0.96**	**0.96**	**2200**
weighted avg	**0.96**	**0.96**	**0.96**	**2200**

Numeric confusion matrix of the GRU (2,113 correct classifications out of 2,200):

Actual \ Pred.	0	1	2	3	4	5	6	7	8	9
0	213	0	0	2	0	1	2	1	0	1
1	0	219	0	0	1	0	0	0	0	0
2	0	0	217	2	0	0	0	0	1	0
3	2	1	3	203	0	0	1	5	5	0
4	0	1	0	0	211	4	0	4	0	0
5	0	0	0	0	2	216	0	1	0	1
6	22	0	9	0	0	0	188	1	0	0
7	0	9	1	0	0	0	0	209	0	1
8	0	0	0	3	0	0	0	0	217	0
9	0	0	0	0	0	0	0	0	0	220

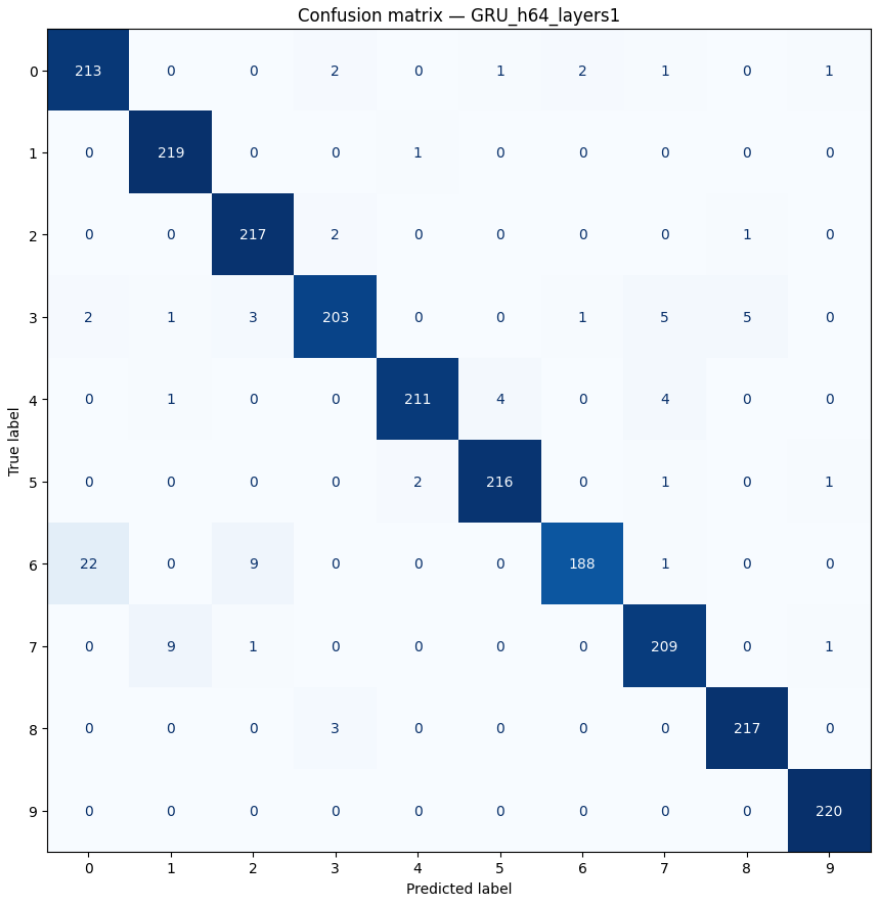


Figure 5. Confusion matrix – GRU_h64_layers1 (test set, 2,200 utterances).

10.2 Best Vanilla RNN: RNN_h128_layers2 – test accuracy 91.91%

The notebook displayed the confusion matrices of the best RNN and best LSTM side by side (Section 9, second figure) but did not print a classification report for them. The per-class precision, recall, and F1 below are therefore computed from the confusion-matrix counts (the same calculation that reproduces the GRU report above exactly).

Class	Precision	Recall	F1-score	Support
0	0.7984	0.9364	0.8619	220
1	0.9860	0.9636	0.9747	220
2	0.9431	0.9045	0.9234	220
3	0.8918	0.9364	0.9135	220
4	0.9692	0.8591	0.9108	220
5	0.9196	0.9364	0.9279	220
6	0.9100	0.8273	0.8667	220
7	0.8861	0.9545	0.9190	220
8	0.9762	0.9318	0.9535	220
9	0.9452	0.9409	0.9431	220
accuracy			**0.9191**	**2200**
macro avg	**0.9226**	**0.9191**	**0.9195**	**2200**
weighted avg	**0.9226**	**0.9191**	**0.9195**	**2200**

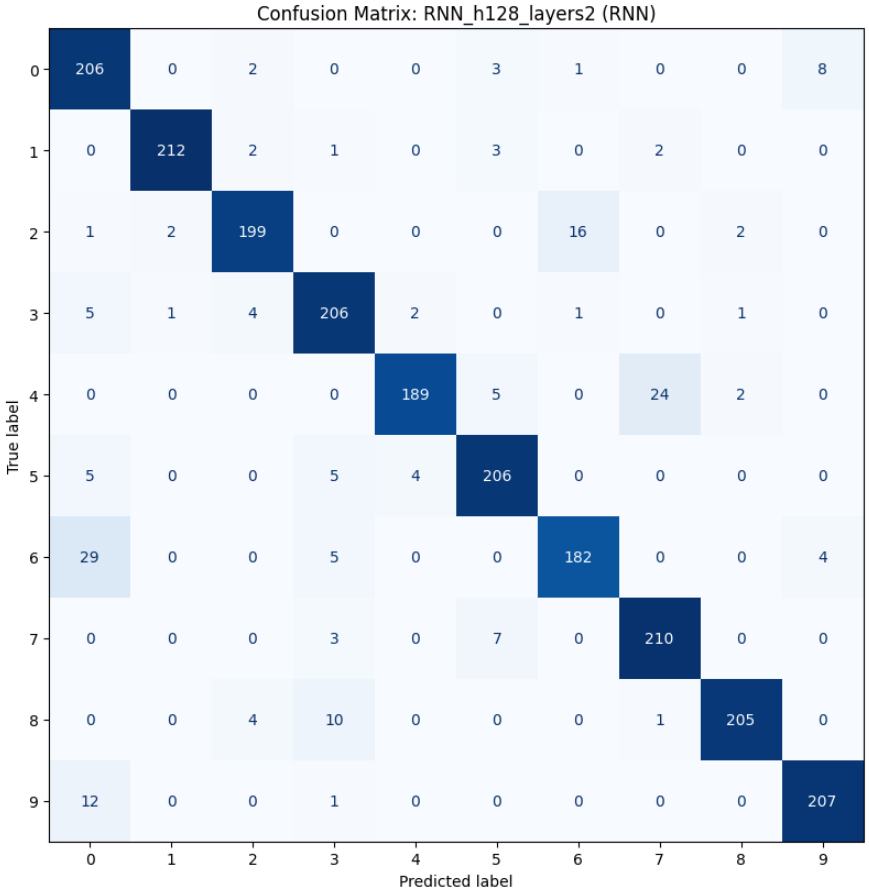


Figure 6. Confusion matrix – best RNN, RNN_h128_layers2 (test set, 2,200 utterances).

10.3 Best LSTM: LSTM_h128_layers2 – test accuracy 97.27%

Class	Precision	Recall	F1-score	Support
0	0.8661	1.0000	0.9283	220
1	0.9955	0.9955	0.9955	220
2	0.9955	0.9955	0.9955	220
3	0.9765	0.9455	0.9607	220
4	0.9862	0.9773	0.9817	220
5	0.9682	0.9682	0.9682	220
6	1.0000	0.9045	0.9499	220
7	0.9767	0.9545	0.9655	220
8	0.9865	1.0000	0.9932	220
9	0.9954	0.9864	0.9909	220
accuracy			**0.9727**	**2200**
macro avg	**0.9747**	**0.9727**	**0.9729**	**2200**
weighted avg	**0.9747**	**0.9727**	**0.9729**	**2200**

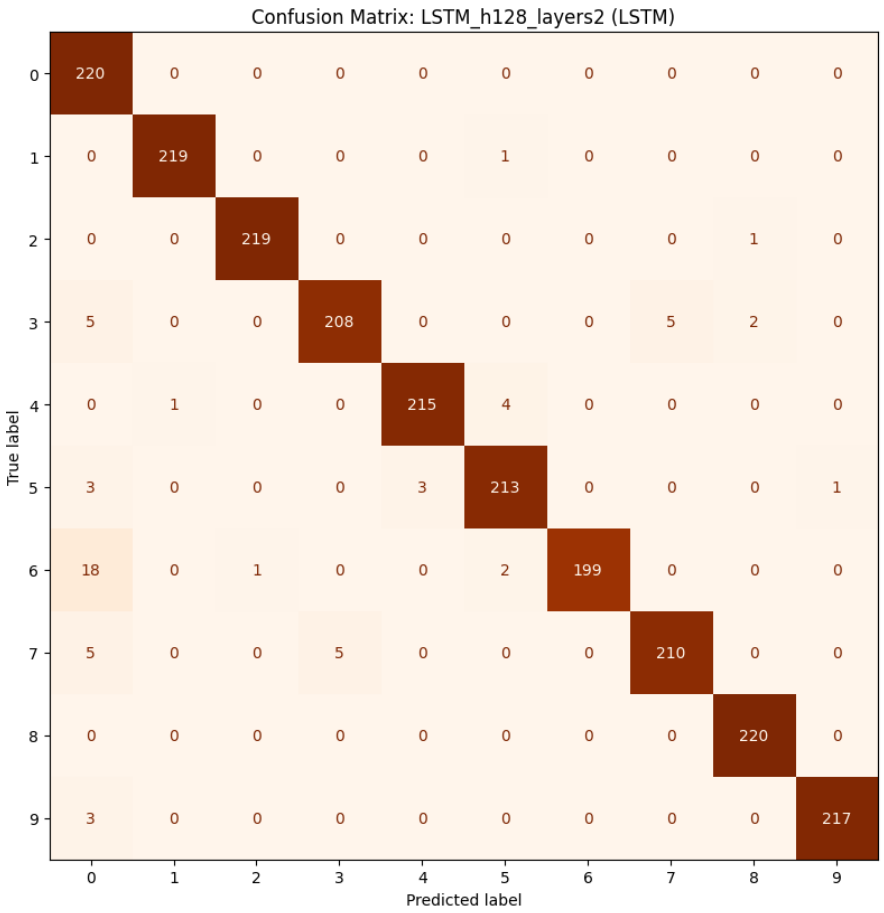


Figure 7. Confusion matrix – best LSTM, LSTM_h128_layers2 (test set, 2,200 utterances).

10.4 Per-class recall of the three best models

Per-class recall computed from the confusion matrices above.

Digit	RNN h128×2	LSTM h128×2	GRU h64×1
0	0.9364	1.0000	0.9682
1	0.9636	0.9955	0.9955
2	0.9045	0.9955	0.9864
3	0.9364	0.9455	0.9227
4	0.8591	0.9773	0.9591
5	0.9364	0.9682	0.9818
6	0.8273	0.9045	0.8545
7	0.9545	0.9545	0.9500
8	0.9318	1.0000	0.9864
9	0.9409	0.9864	1.0000

The largest off-diagonal counts are: for the RNN, digit 6 predicted as 0 (29), digit 4 as 7 (24), digit 2 as 6 (16), digit 9 as 0 (12), and digit 8 as 3 (10); for the LSTM, digit 6 predicted as 0 (18) and four 5-count confusions (3 as 0, 3 as 7, 7 as 0, and 7 as 3); for the GRU, digit 6 predicted as 0 (22), digit 6 as 2 (9), and digit 7 as 1 (9). Digit 6 is the lowest-recall class for all three of the best RNN (0.8273), LSTM (0.9045), and GRU (0.8545), and its most frequent error is being predicted as digit 0 in all three models.

11. Comparative Observations and Empirical Analysis (e)

11.1 Performance

By validation accuracy the ranking is GRU_h64_layers1 (99.39%), LSTM_h128_layers2 (99.29%), LSTM_h64_layers1 (98.79%), RNN_h128_layers2 (96.46%), and RNN_h64_layers1 (96.36%). The gap between the first two is 1 of the 990 validation utterances. By test accuracy the ranking is LSTM_h128_layers2 (97.27%), GRU_h64_layers1 (96.05%), LSTM_h64_layers1 (94.82%), RNN_h128_layers2 (91.91%), and RNN_h64_layers1 (88.86%): only the order of the top two changes.

The gated models outperform the vanilla RNN in every like-for-like comparison. With 64 units and one layer the LSTM improves test accuracy over the RNN by 5.95 percentage points and the GRU by 7.18; with 128 units and two layers the LSTM improves on the RNN by 5.36 points. Test loss shows the same pattern (0.3445 for the best RNN, 0.1785 for the GRU, 0.1197 for the best LSTM). The larger two-layer models were better than the one-layer models for both RNN and LSTM, although size, depth, and dropout changed together in this comparison.

For every configuration the test accuracy is below the validation accuracy; the difference ranges from 2.02 points (best LSTM) to 7.50 points (RNN_h64_layers1). This is consistent with the test speakers being different from the training speakers, whereas the validation set is a random split of the training file.

11.2 Convergence

Under the assignment's rule (change in average training loss below 1e-4 between consecutive epochs), LSTM_h128_layers2 converged at epoch 23 and GRU_h64_layers1 at epoch 31. The other three runs ended through the validation-loss safety stop after 12 epochs without improvement, at epochs 33 (LSTM_h64_layers1), 35 (RNN_h128_layers2), and 51 (RNN_h64_layers1), so the convergence rule itself had not been met in those runs.

Training loss falls fastest for the gated models. The LSTM_h128_layers2 training loss drops from 1.3410 in epoch 1 to 0.0005 at its final epoch, the GRU from 1.8996 to 0.0036, and the one-layer LSTM from 1.8711 to 0.0018, whereas the two RNNs finish at 0.1130 (h128 × 2) and 0.0733 (h64 × 1) after starting from 1.4534 and 1.8326. Validation accuracy first reaches 95% at epoch 2 for LSTM_h128_layers2 and epoch 5 for the GRU, compared with epoch 34 for RNN_h64_layers1. The RNN curves are also noisier: for example the RNN_h64_layers1 validation loss rises from 0.7698 (epoch 6) to 0.8661 (epoch 7) before continuing to fall.

11.3 Padding, packing, and gating

All models used the same padded and packed pipeline, so the differences above come from the recurrent cell and the configuration, not from the handling of sequence length. Utterances are up to 93 frames long. LSTM and GRU gates are designed to help preserve information across many time steps, which is consistent with their faster and lower training loss here; however, the notebook did not run a gradient-flow diagnostic or a padding/packing ablation, so these mechanisms are not measured directly by these experiments.

11.4 Limitations

- Each configuration was trained once with a single seed (42); differences of about one validation utterance (for example GRU versus the best LSTM) should not be over-interpreted.
- Only one GRU configuration was run (a bonus model), so the GRU is compared with the other cells mainly at 64 units and one layer.
- Hidden size, number of layers, and dropout were changed together between the “small” and “large” configurations.

- Three of the five runs ended through the validation-loss safety stop rather than the 1e-4 convergence rule, and the maximum of 200 epochs was never reached.
- The train accuracy and loss in Section 8 are re-evaluated with the restored best-validation-loss weights, so they can differ slightly from the final-epoch values in the training log.
- The notebook printed a classification report only for the validation-selected GRU; the RNN and LSTM reports in Sections 10.2 and 10.3 are computed from their confusion matrices.
- The validation set is a random split of the training file, and speaker identities are not available in the loaded data, so speaker-level generalization is assessed only through the separate test file.

12. Conclusion

Across five recurrent configurations on the Spoken Arabic Digit data, the validation-selected model is GRU_h64_layers1, with 99.39% validation accuracy and 96.05% test accuracy. LSTM_h128_layers2 is essentially tied on validation (99.29%) and has the highest test accuracy (97.27%) of all configurations. The best vanilla RNN reaches 91.91% test accuracy, so the gated architectures are clearly stronger in this experiment and converge faster. Digit 6, most often mistaken for digit 0, is the hardest class for all three best models. Because every configuration was run once, a multi-seed repeat would be needed to separate the GRU and LSTM reliably.

Appendix A. Per-Epoch Training Log of All Experiments

Verbatim notebook output of the hyperparameter experiment cell (Section 7 of the notebook): average training and validation loss and accuracy for every epoch of every configuration.

```
================================================================================
Experiment 1/5: RNN_h64_layers1
================================================================================
Epoch 001 | train loss=1.832552 | val loss=1.463994 | train acc=0.3294 | val acc=0.4515
Epoch 002 | train loss=1.299047 | val loss=1.196285 | train acc=0.5189 | val acc=0.5364
Epoch 003 | train loss=1.132332 | val loss=1.091788 | train acc=0.5677 | val acc=0.5707
Epoch 004 | train loss=1.041077 | val loss=1.002333 | train acc=0.6000 | val acc=0.6051
Epoch 005 | train loss=0.937362 | val loss=0.922282 | train acc=0.6367 | val acc=0.6424
Epoch 006 | train loss=0.829879 | val loss=0.769794 | train acc=0.6854 | val acc=0.7101
Epoch 007 | train loss=0.732045 | val loss=0.866126 | train acc=0.7193 | val acc=0.6848
Epoch 008 | train loss=0.674092 | val loss=0.660198 | train acc=0.7376 | val acc=0.7374
Epoch 009 | train loss=0.650697 | val loss=0.604803 | train acc=0.7444 | val acc=0.7636
Epoch 010 | train loss=0.608383 | val loss=0.649358 | train acc=0.7528 | val acc=0.7515
Epoch 011 | train loss=0.568504 | val loss=0.595864 | train acc=0.7729 | val acc=0.7697
Epoch 012 | train loss=0.525201 | val loss=0.559619 | train acc=0.7857 | val acc=0.7798
Epoch 013 | train loss=0.517053 | val loss=0.530232 | train acc=0.7904 | val acc=0.7606
Epoch 014 | train loss=0.492317 | val loss=0.517927 | train acc=0.8037 | val acc=0.8000
Epoch 015 | train loss=0.486894 | val loss=0.492866 | train acc=0.8062 | val acc=0.7990
Epoch 016 | train loss=0.453606 | val loss=0.441715 | train acc=0.8196 | val acc=0.8313
Epoch 017 | train loss=0.431305 | val loss=0.461127 | train acc=0.8301 | val acc=0.8172
Epoch 018 | train loss=0.412783 | val loss=0.420047 | train acc=0.8483 | val acc=0.8374
Epoch 019 | train loss=0.387928 | val loss=0.431433 | train acc=0.8570 | val acc=0.8525
Epoch 020 | train loss=0.351183 | val loss=0.431137 | train acc=0.8781 | val acc=0.8343
Epoch 021 | train loss=0.335618 | val loss=0.363235 | train acc=0.8790 | val acc=0.8879
Epoch 022 | train loss=0.302709 | val loss=0.341638 | train acc=0.9061 | val acc=0.9061
Epoch 023 | train loss=0.264658 | val loss=0.272316 | train acc=0.9189 | val acc=0.9192
Epoch 024 | train loss=0.273726 | val loss=0.292307 | train acc=0.9198 | val acc=0.9111
Epoch 025 | train loss=0.231162 | val loss=0.233697 | train acc=0.9357 | val acc=0.9404
Epoch 026 | train loss=0.204179 | val loss=0.215711 | train acc=0.9435 | val acc=0.9444
Epoch 027 | train loss=0.187085 | val loss=0.269922 | train acc=0.9455 | val acc=0.9283
Epoch 028 | train loss=0.174615 | val loss=0.221205 | train acc=0.9531 | val acc=0.9414
Epoch 029 | train loss=0.141707 | val loss=0.254018 | train acc=0.9620 | val acc=0.9333
Epoch 030 | train loss=0.180062 | val loss=0.236703 | train acc=0.9540 | val acc=0.9384
Epoch 031 | train loss=0.170872 | val loss=0.238997 | train acc=0.9542 | val acc=0.9444
Epoch 032 | train loss=0.150567 | val loss=0.217306 | train acc=0.9578 | val acc=0.9434
Epoch 033 | train loss=0.129127 | val loss=0.228106 | train acc=0.9642 | val acc=0.9394
Epoch 034 | train loss=0.177011 | val loss=0.180634 | train acc=0.9556 | val acc=0.9535
Epoch 035 | train loss=0.119195 | val loss=0.187966 | train acc=0.9695 | val acc=0.9505
Epoch 036 | train loss=0.129442 | val loss=0.190193 | train acc=0.9670 | val acc=0.9556
Epoch 037 | train loss=0.118733 | val loss=0.191246 | train acc=0.9693 | val acc=0.9525
Epoch 038 | train loss=0.119203 | val loss=0.173000 | train acc=0.9690 | val acc=0.9596
Epoch 039 | train loss=0.110862 | val loss=0.130959 | train acc=0.9715 | val acc=0.9636
Epoch 040 | train loss=0.109574 | val loss=0.150225 | train acc=0.9752 | val acc=0.9636
Epoch 041 | train loss=0.091796 | val loss=0.163966 | train acc=0.9766 | val acc=0.9576
Epoch 042 | train loss=0.112356 | val loss=0.155847 | train acc=0.9720 | val acc=0.9606
Epoch 043 | train loss=0.109744 | val loss=0.156616 | train acc=0.9729 | val acc=0.9626
Epoch 044 | train loss=0.127003 | val loss=0.238748 | train acc=0.9679 | val acc=0.9515
Epoch 045 | train loss=0.098594 | val loss=0.151451 | train acc=0.9766 | val acc=0.9616
Epoch 046 | train loss=0.115786 | val loss=0.221693 | train acc=0.9718 | val acc=0.9505
Epoch 047 | train loss=0.111834 | val loss=0.182842 | train acc=0.9758 | val acc=0.9616
Epoch 048 | train loss=0.079367 | val loss=0.180179 | train acc=0.9820 | val acc=0.9636
Epoch 049 | train loss=0.072388 | val loss=0.137534 | train acc=0.9841 | val acc=0.9657
Epoch 050 | train loss=0.091406 | val loss=0.164527 | train acc=0.9777 | val acc=0.9677
Epoch 051 | train loss=0.073326 | val loss=0.174196 | train acc=0.9836 | val acc=0.9657
Stopping: early stop: validation loss did not improve for 12 epochs

================================================================================
Experiment 2/5: RNN_h128_layers2
================================================================================
Epoch 001 | train loss=1.453389 | val loss=1.132693 | train acc=0.4567 | val acc=0.5919
Epoch 002 | train loss=0.923886 | val loss=0.706361 | train acc=0.6747 | val acc=0.7525
Epoch 003 | train loss=0.720580 | val loss=0.601509 | train acc=0.7524 | val acc=0.8071
Epoch 004 | train loss=0.635087 | val loss=0.592817 | train acc=0.7865 | val acc=0.8101
Epoch 005 | train loss=0.544237 | val loss=0.688956 | train acc=0.8242 | val acc=0.7677
Epoch 006 | train loss=0.476490 | val loss=0.378191 | train acc=0.8465 | val acc=0.8929
Epoch 007 | train loss=0.383045 | val loss=0.508997 | train acc=0.8850 | val acc=0.8354
Epoch 008 | train loss=0.344238 | val loss=0.359373 | train acc=0.8973 | val acc=0.9040
Epoch 009 | train loss=0.338176 | val loss=0.282453 | train acc=0.8993 | val acc=0.9141
Epoch 010 | train loss=0.251463 | val loss=0.236414 | train acc=0.9296 | val acc=0.9384
Epoch 011 | train loss=0.245163 | val loss=0.265542 | train acc=0.9301 | val acc=0.9364
Epoch 012 | train loss=0.305176 | val loss=0.250387 | train acc=0.9135 | val acc=0.9273
Epoch 013 | train loss=0.197679 | val loss=0.332985 | train acc=0.9463 | val acc=0.9162
Epoch 014 | train loss=0.221347 | val loss=0.237179 | train acc=0.9399 | val acc=0.9384
Epoch 015 | train loss=0.209999 | val loss=0.228486 | train acc=0.9444 | val acc=0.9394
Epoch 016 | train loss=0.181674 | val loss=0.289019 | train acc=0.9497 | val acc=0.9273
Epoch 017 | train loss=0.200279 | val loss=0.206899 | train acc=0.9449 | val acc=0.9394
Epoch 018 | train loss=0.195567 | val loss=0.272099 | train acc=0.9456 | val acc=0.9242
Epoch 019 | train loss=0.139444 | val loss=0.152973 | train acc=0.9617 | val acc=0.9646
Epoch 020 | train loss=0.148139 | val loss=0.197343 | train acc=0.9601 | val acc=0.9475
Epoch 021 | train loss=0.126422 | val loss=0.175224 | train acc=0.9635 | val acc=0.9626
Epoch 022 | train loss=0.119904 | val loss=0.185981 | train acc=0.9663 | val acc=0.9535
Epoch 023 | train loss=0.164528 | val loss=0.133457 | train acc=0.9544 | val acc=0.9646
Epoch 024 | train loss=0.132893 | val loss=0.189912 | train acc=0.9660 | val acc=0.9475
```

```
Epoch 025 | train loss=0.180116 | val loss=0.224742 | train acc=0.9453 | val acc=0.9414
Epoch 026 | train loss=0.122511 | val loss=0.140457 | train acc=0.9701 | val acc=0.9636
Epoch 027 | train loss=0.108754 | val loss=0.172628 | train acc=0.9722 | val acc=0.9525
Epoch 028 | train loss=0.138436 | val loss=0.183382 | train acc=0.9642 | val acc=0.9535
Epoch 029 | train loss=0.142102 | val loss=0.143340 | train acc=0.9622 | val acc=0.9657
Epoch 030 | train loss=0.114424 | val loss=0.153395 | train acc=0.9704 | val acc=0.9566
Epoch 031 | train loss=0.135262 | val loss=0.202084 | train acc=0.9633 | val acc=0.9495
Epoch 032 | train loss=0.115527 | val loss=0.204195 | train acc=0.9722 | val acc=0.9444
Epoch 033 | train loss=0.105990 | val loss=0.155719 | train acc=0.9759 | val acc=0.9636
Epoch 034 | train loss=0.106225 | val loss=0.198928 | train acc=0.9717 | val acc=0.9465
Epoch 035 | train loss=0.113029 | val loss=0.143334 | train acc=0.9717 | val acc=0.9657
Stopping: early stop: validation loss did not improve for 12 epochs

================================================================================
Experiment 3/5: LSTM_h64_layers1
================================================================================
Epoch 001 | train loss=1.871056 | val loss=1.258770 | train acc=0.3332 | val acc=0.5737
Epoch 002 | train loss=0.906004 | val loss=0.650042 | train acc=0.6991 | val acc=0.8141
Epoch 003 | train loss=0.482558 | val loss=0.323745 | train acc=0.8736 | val acc=0.9333
Epoch 004 | train loss=0.261818 | val loss=0.236232 | train acc=0.9396 | val acc=0.9455
Epoch 005 | train loss=0.180382 | val loss=0.159092 | train acc=0.9601 | val acc=0.9646
Epoch 006 | train loss=0.147021 | val loss=0.145227 | train acc=0.9638 | val acc=0.9657
Epoch 007 | train loss=0.115589 | val loss=0.110498 | train acc=0.9724 | val acc=0.9727
Epoch 008 | train loss=0.094213 | val loss=0.088811 | train acc=0.9793 | val acc=0.9788
Epoch 009 | train loss=0.074186 | val loss=0.071312 | train acc=0.9829 | val acc=0.9808
Epoch 010 | train loss=0.054878 | val loss=0.113241 | train acc=0.9881 | val acc=0.9677
Epoch 011 | train loss=0.055642 | val loss=0.071920 | train acc=0.9873 | val acc=0.9808
Epoch 012 | train loss=0.055443 | val loss=0.085421 | train acc=0.9868 | val acc=0.9778
Epoch 013 | train loss=0.043953 | val loss=0.076396 | train acc=0.9909 | val acc=0.9778
Epoch 014 | train loss=0.032549 | val loss=0.096317 | train acc=0.9929 | val acc=0.9747
Epoch 015 | train loss=0.047176 | val loss=0.061491 | train acc=0.9870 | val acc=0.9838
Epoch 016 | train loss=0.029662 | val loss=0.060849 | train acc=0.9932 | val acc=0.9828
Epoch 017 | train loss=0.026277 | val loss=0.092935 | train acc=0.9938 | val acc=0.9747
Epoch 018 | train loss=0.038133 | val loss=0.074239 | train acc=0.9904 | val acc=0.9818
Epoch 019 | train loss=0.020848 | val loss=0.054272 | train acc=0.9950 | val acc=0.9838
Epoch 020 | train loss=0.031609 | val loss=0.056130 | train acc=0.9927 | val acc=0.9838
Epoch 021 | train loss=0.016423 | val loss=0.040715 | train acc=0.9968 | val acc=0.9879
Epoch 022 | train loss=0.014736 | val loss=0.046265 | train acc=0.9961 | val acc=0.9869
Epoch 023 | train loss=0.012918 | val loss=0.061536 | train acc=0.9975 | val acc=0.9879
Epoch 024 | train loss=0.021401 | val loss=0.086937 | train acc=0.9952 | val acc=0.9768
Epoch 025 | train loss=0.015400 | val loss=0.070271 | train acc=0.9964 | val acc=0.9808
Epoch 026 | train loss=0.012566 | val loss=0.043350 | train acc=0.9973 | val acc=0.9889
Epoch 027 | train loss=0.008943 | val loss=0.048131 | train acc=0.9982 | val acc=0.9889
Epoch 028 | train loss=0.004550 | val loss=0.043566 | train acc=0.9996 | val acc=0.9899
Epoch 029 | train loss=0.003229 | val loss=0.043644 | train acc=0.9996 | val acc=0.9899
Epoch 030 | train loss=0.002689 | val loss=0.042893 | train acc=0.9998 | val acc=0.9899
Epoch 031 | train loss=0.002351 | val loss=0.041815 | train acc=1.0000 | val acc=0.9899
Epoch 032 | train loss=0.001970 | val loss=0.041467 | train acc=1.0000 | val acc=0.9909
Epoch 033 | train loss=0.001756 | val loss=0.042897 | train acc=1.0000 | val acc=0.9909
Stopping: early stop: validation loss did not improve for 12 epochs

================================================================================
Experiment 4/5: LSTM_h128_layers2
================================================================================
Epoch 001 | train loss=1.341008 | val loss=0.475723 | train acc=0.5374 | val acc=0.8717
Epoch 002 | train loss=0.273322 | val loss=0.173788 | train acc=0.9200 | val acc=0.9515
Epoch 003 | train loss=0.166139 | val loss=0.093435 | train acc=0.9537 | val acc=0.9717
Epoch 004 | train loss=0.083908 | val loss=0.080833 | train acc=0.9752 | val acc=0.9717
Epoch 005 | train loss=0.058766 | val loss=0.104717 | train acc=0.9829 | val acc=0.9667
Epoch 006 | train loss=0.042750 | val loss=0.038689 | train acc=0.9882 | val acc=0.9889
Epoch 007 | train loss=0.028108 | val loss=0.044735 | train acc=0.9925 | val acc=0.9879
Epoch 008 | train loss=0.027950 | val loss=0.035197 | train acc=0.9913 | val acc=0.9899
Epoch 009 | train loss=0.021392 | val loss=0.041227 | train acc=0.9945 | val acc=0.9889
Epoch 010 | train loss=0.015810 | val loss=0.048527 | train acc=0.9961 | val acc=0.9828
Epoch 011 | train loss=0.023000 | val loss=0.030008 | train acc=0.9941 | val acc=0.9919
Epoch 012 | train loss=0.026287 | val loss=0.025237 | train acc=0.9922 | val acc=0.9929
Epoch 013 | train loss=0.023958 | val loss=0.026334 | train acc=0.9934 | val acc=0.9929
Epoch 014 | train loss=0.010112 | val loss=0.025912 | train acc=0.9968 | val acc=0.9949
Epoch 015 | train loss=0.018044 | val loss=0.048453 | train acc=0.9952 | val acc=0.9869
Epoch 016 | train loss=0.006754 | val loss=0.033630 | train acc=0.9984 | val acc=0.9899
Epoch 017 | train loss=0.002166 | val loss=0.026752 | train acc=0.9998 | val acc=0.9929
Epoch 018 | train loss=0.001327 | val loss=0.025179 | train acc=0.9998 | val acc=0.9929
Epoch 019 | train loss=0.001017 | val loss=0.027038 | train acc=0.9998 | val acc=0.9929
Epoch 020 | train loss=0.000849 | val loss=0.027488 | train acc=0.9998 | val acc=0.9929
Epoch 021 | train loss=0.000712 | val loss=0.026285 | train acc=1.0000 | val acc=0.9929
Epoch 022 | train loss=0.000603 | val loss=0.025652 | train acc=1.0000 | val acc=0.9929
Epoch 023 | train loss=0.000518 | val loss=0.026084 | train acc=1.0000 | val acc=0.9929
Stopping: convergence rule met: |loss change|=8.49001e-05 < 0.0001

================================================================================
Experiment 5/5: GRU_h64_layers1
================================================================================
Epoch 001 | train loss=1.899566 | val loss=1.356351 | train acc=0.3160 | val acc=0.5535
Epoch 002 | train loss=0.994807 | val loss=0.728245 | train acc=0.6633 | val acc=0.7677
Epoch 003 | train loss=0.576818 | val loss=0.451203 | train acc=0.8237 | val acc=0.8869
Epoch 004 | train loss=0.349365 | val loss=0.269917 | train acc=0.9066 | val acc=0.9414
Epoch 005 | train loss=0.232218 | val loss=0.187135 | train acc=0.9421 | val acc=0.9566
Epoch 006 | train loss=0.159013 | val loss=0.154015 | train acc=0.9620 | val acc=0.9596
Epoch 007 | train loss=0.120927 | val loss=0.108752 | train acc=0.9697 | val acc=0.9717
Epoch 008 | train loss=0.083807 | val loss=0.113273 | train acc=0.9820 | val acc=0.9707
```

```
Epoch 009 | train loss=0.086176 | val loss=0.105609 | train acc=0.9795 | val acc=0.9717
Epoch 010 | train loss=0.058150 | val loss=0.091518 | train acc=0.9852 | val acc=0.9747
Epoch 011 | train loss=0.050163 | val loss=0.057830 | train acc=0.9895 | val acc=0.9818
Epoch 012 | train loss=0.038642 | val loss=0.055912 | train acc=0.9913 | val acc=0.9869
Epoch 013 | train loss=0.036727 | val loss=0.042650 | train acc=0.9914 | val acc=0.9848
Epoch 014 | train loss=0.025738 | val loss=0.040704 | train acc=0.9945 | val acc=0.9838
Epoch 015 | train loss=0.026382 | val loss=0.055988 | train acc=0.9943 | val acc=0.9818
Epoch 016 | train loss=0.025857 | val loss=0.039468 | train acc=0.9936 | val acc=0.9838
Epoch 017 | train loss=0.017692 | val loss=0.035010 | train acc=0.9971 | val acc=0.9879
Epoch 018 | train loss=0.014678 | val loss=0.030202 | train acc=0.9984 | val acc=0.9889
Epoch 019 | train loss=0.018684 | val loss=0.043453 | train acc=0.9970 | val acc=0.9859
Epoch 020 | train loss=0.029674 | val loss=0.045263 | train acc=0.9934 | val acc=0.9869
Epoch 021 | train loss=0.029891 | val loss=0.050496 | train acc=0.9923 | val acc=0.9848
Epoch 022 | train loss=0.020737 | val loss=0.041174 | train acc=0.9941 | val acc=0.9869
Epoch 023 | train loss=0.010412 | val loss=0.027492 | train acc=0.9988 | val acc=0.9949
Epoch 024 | train loss=0.009585 | val loss=0.023826 | train acc=0.9986 | val acc=0.9929
Epoch 025 | train loss=0.006564 | val loss=0.027782 | train acc=0.9993 | val acc=0.9929
Epoch 026 | train loss=0.007981 | val loss=0.026660 | train acc=0.9989 | val acc=0.9919
Epoch 027 | train loss=0.005263 | val loss=0.024769 | train acc=0.9993 | val acc=0.9929
Epoch 028 | train loss=0.004627 | val loss=0.024254 | train acc=0.9993 | val acc=0.9929
Epoch 029 | train loss=0.004093 | val loss=0.022596 | train acc=0.9995 | val acc=0.9929
Epoch 030 | train loss=0.003615 | val loss=0.018951 | train acc=0.9998 | val acc=0.9939
Epoch 031 | train loss=0.003576 | val loss=0.020070 | train acc=0.9998 | val acc=0.9939
Stopping: convergence rule met: |loss change|=3.88723e-05 < 0.0001
```